Autómata del tres en raya contra el servidor

Caso práctico

Carlos quiere un tres en raya en el que el contrincante sea el servidor. Al dibujar el autómata se atasca: después de que el jugador pulse una casilla, ¿qué evento hace jugar a la máquina? El navegador no envía nada más. **María** le enseña un truco de los autómatas de protocolos: un **evento interno**, que no viene de fuera sino que lo produce el propio programa, y que se atiende en la misma petición.

1.- Enunciado

Programa en PHP un **tres en raya** contra el servidor:

- Antes de empezar se elige la **ficha** (X u O) y **quién empieza** (el jugador o la máquina).
- Cada casilla libre es un botón. Tras la jugada del jugador, si no ha ganado ni se ha llenado el tablero, **juega la máquina** en la misma respuesta.
- La máquina juega así, en este orden: si puede **ganar**, gana; si no, **tapa** la jugada ganadora del jugador; si no, el **centro**; si no, una **esquina** (0, 2, 6, 8); si no, un **lado**.
- La partida termina con **victoria**, **derrota** o **empate**, y se muestra un **marcador** (ganadas, perdidas y empates) que se conserva entre partidas. *Abandonar* cuenta como derrota.
- Al terminar: *Revancha* (misma configuración) o *Cambiar de ficha* (volver a la configuración conservando el marcador).
- La partida viaja en JSON en un campo oculto y se comprueba que es coherente (fichas válidas y que el número de fichas de cada uno cuadra con quién empezó).

2.- Análisis

- **Eventos entrantes.** Seis que vienen del navegador (Carga, Empezar, Casilla, Revancha, Abandonar, Cambiar de ficha) y **uno interno**: *turno de la máquina*.
- **Estados.** Siete: configuración, turno del jugador, **turno de la máquina**, ganada, perdida y empate (más ST0). ST3, el turno de la máquina, es un estado **transitorio**: nunca se envía al navegador. Cuando una transición termina en ST3, el programa atiende enseguida el evento interno (T4), que deja el autómata en ST2, ST5 o ST6.
- **Predicados de la jugada.** PL5 (la jugada del jugador gana) y PL6 (llena el tablero) se calculan **antes** de poner la ficha, sobre el tablero con la ficha "imaginada". Igual PL7 y PL8 para la máquina.
- **El orden importa.** Si una jugada gana y a la vez llena el tablero (la novena ficha), es victoria: PL5 se comprueba antes que PL6.
- **Coherencia (PL3).** Un tablero `XXX------` con el jugador X que empezaba es imposible (tres X y ninguna O): la partida se ha manipulado.

3.- Diagrama de estado-transición ampliado

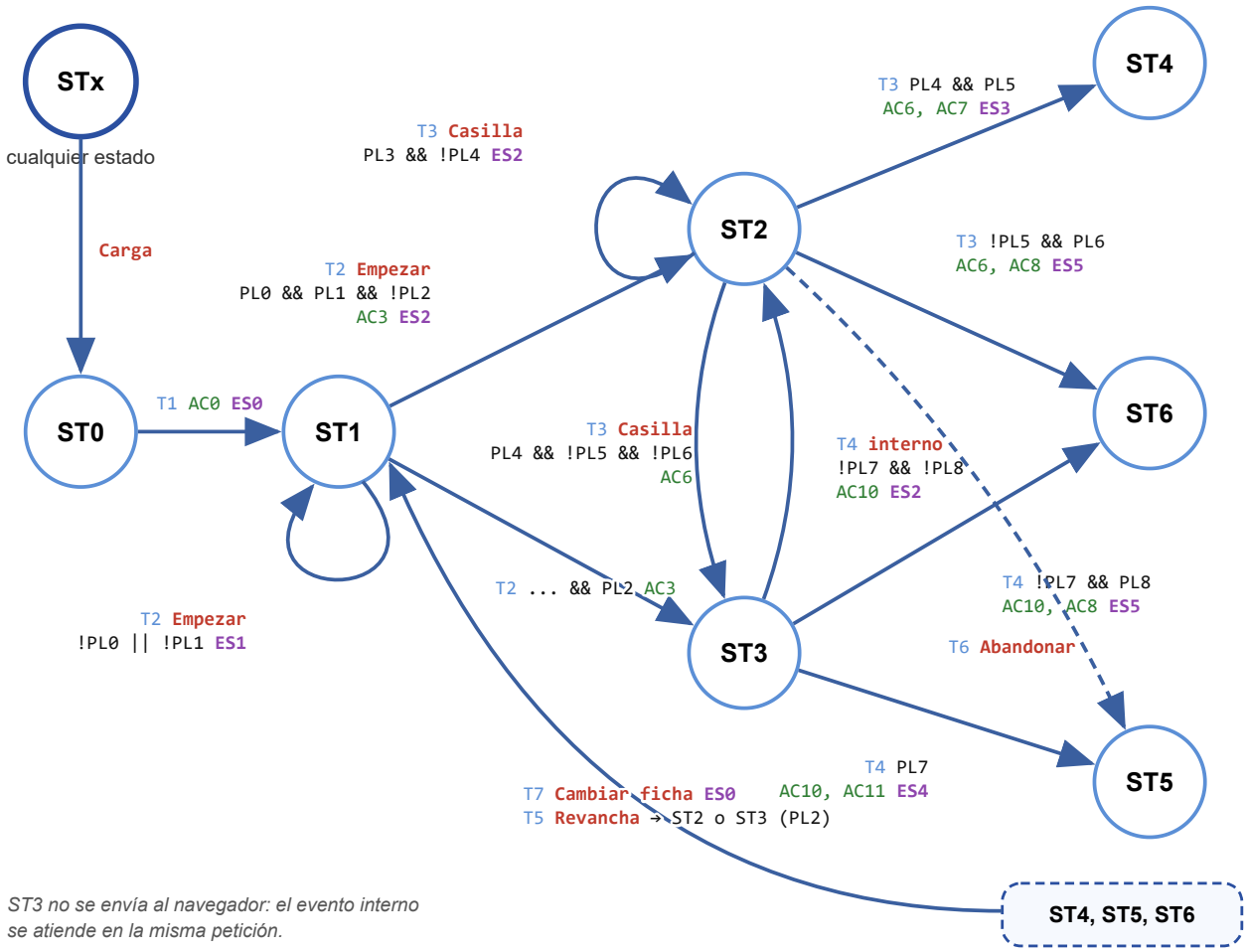


Figura 1. Diagrama del tres en raya (ST3 es transitorio: el evento interno se atiende en la misma petición)

4.- Autómata

4.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Carga	página sin parámetros
Evento entrante	EE1	Empezar	enviar la configuración
Evento entrante	EE2	Casilla	`casilla:0` ... `casilla:8`
Evento entrante	EE3	Revancha	otra partida con la misma configuración
Evento entrante	EE4	Abandonar	
Evento entrante	EE5	Interno	turno de la máquina (lo genera el programa, no el navegador)
Evento entrante	EE6	Configurar	"Cambiar de ficha"
Evento saliente	ES0 / ES1	configuracion	ficha, quién empieza y marcador; ES1 con aviso
Evento saliente	ES2	tablero	le toca al jugador (con aviso si la casilla estaba ocupada)
Evento saliente	ES3, ES4, ES5	ganada, perdida, empate	tablero sin botones, marcador, Revancha y Cambiar de ficha
Estados	ST0–ST6	—	Carga, Configuracion, TurnoJugador, TurnoMaquina, Ganada, Perdida, Empate
Variable de estado	VE0	`$estado`	(nunca vale ST3 en la página)
Variable de estado	VE1	`$p`	JSON: tablero (9 caracteres X, O o -), ficha, empieza, marcador
Predicado	PL0, PL1	configuración válida	ficha X u O; empieza "jugador" o "maquina"
Predicado	PL2	empieza la máquina	
Predicado	PL3	partida coherente	forma del JSON y número de fichas
Predicado	PL4	casilla libre	entre 0 y 8 y con "-"
Predicado	PL5	gana el jugador	la jugada hace tres en raya
Predicado	PL6	se llena (jugador)	quedaba una sola casilla libre
Predicado	PL7	gana la máquina	su jugada hace tres en raya
Predicado	PL8	se llena (máquina)	quedaba una sola casilla libre
Acción	AC0	marcador a cero	
Acción	AC1	recuperar	el marcador (T2, T7) o la partida (T5)

Tipo	Abreviatura	Nombre	Significado
Acción	AC2	leer la configuración	
Acción	AC3	tablero vacío	`---------`
Acción	AC4	recuperar la partida	y comprobarla (PL3)
Acción	AC5	leer la casilla	
Acción	AC6	poner la ficha del jugador	
Acción	AC7, AC8, AC11	ganadas++, empates++, perdidas++	
Acción	AC9	elegir la jugada de la máquina	ganar, tapar, centro, esquinas, lados
Acción	AC10	poner la ficha de la máquina	
Acción	AC12	guardar	la partida en el campo oculto

4.2.- Tabla de eventos-estados ampliada

	Carga	Empezar	Casilla	Revancha	Abandonar	Interno	Configurar
ST0	T1	T1	T1	T1	T1	---	T1
ST1	T1	T2	---	---	---	---	---
ST2	T1	---	T3	---	T6	---	---
ST3	---	---	---	---	---	T4	---
ST4	T1	---	---	T5	---	---	T7
ST5	T1	---	---	T5	---	---	T7
ST6	T1	---	---	T5	---	---	T7

La fila de ST3 es la clave: en ese estado **solo** puede ocurrir el evento interno, y la columna "Interno" solo tiene una celda.

4.3.- Tabla de transiciones

#	AEs antes	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	AC0	ES0 [AC12]
T2	AC1, AC2	`!PL0 || !PL1`	ST1	---	ES1 [AC12]
T2	AC1, AC2	`PL0 && PL1 && !PL2`	ST2	AC3	ES2 [AC12]
T2	AC1, AC2	`PL0 && PL1 && PL2`	ST3	AC3	(evento interno)
T3	AC4, AC5	`!PL3`	ST1	---	ES1 [AC12]
T3	AC4, AC5	`PL3 && !PL4`	ST2	---	ES2 "no está libre" [AC12]
T3	AC4, AC5	`PL3 && PL4 && PL5`	ST4	AC6, AC7	ES3 [AC12]
T3	AC4, AC5	`PL3 && PL4 && !PL5 && PL6`	ST6	AC6, AC8	ES5 [AC12]
T3	AC4, AC5	`PL3 && PL4 && !PL5 && !PL6`	ST3	AC6	(evento interno)
T4	AC9	`PL7`	ST5	AC10, AC11	ES4 [AC12]
T4	AC9	`!PL7 && PL8`	ST6	AC10, AC8	ES5 [AC12]
T4	AC9	`!PL7 && !PL8`	ST2	AC10	ES2 [AC12]
T5	AC1	`!PL3`	ST1	---	ES1 [AC12]
T5	AC1	`PL3 && !PL2`	ST2	AC3	ES2 [AC12]
T5	AC1	`PL3 && PL2`	ST3	AC3	(evento interno)
T6	AC4	`!PL3` / `PL3`	ST1 / ST5	--- / AC11	ES1 / ES4 "ha abandonado" [AC12]
T7	AC1	---	ST1	---	ES0 (con el marcador) [AC12]

4.4.- Comprobación de predicados

T3:

PL3	PL4	PL5	PL6	`!PL3`	ocupada	gana	empate	juega la máquina
0	x	x	x	1	0	0	0	0
1	0	x	x	0	1	0	0	0
1	1	1	x	0	0	1	0	0
1	1	0	1	0	0	0	1	0
1	1	0	0	0	0	0	0	1

T4:

PL7	PL8	pierde	empate	sigue
1	x	1	0	0
0	1	0	1	0
0	0	0	0	1

5.- Del autómata al código

El `switch` va dentro de un bucle `do ... while`. Si una transición termina en ST3, el bucle vuelve a entrar con la transición del evento interno (T4). Así el programa sigue la tabla de eventos-estados también para los eventos que no vienen del navegador:

```
do {
    $interno = false;
    switch ($transicion) {
        // ... T1, T2, T3 (pueden terminar en ST3)
        case 4:                                             // ---- T4 (ST3, Interno: turno de
la máquina)
            $maquina = contraria($p['ficha']);
            $jugada = jugadaMaquina($p['tablero'], $maquina);   // AEs PRE: AC9
            $pl7 = haceTresEnRaya($p['tablero'], $jugada, $maquina);
            $pl8 = seLlena($p['tablero']);
            $p['tablero'][$jugada] = $maquina;              // AC10: poner la ficha de la
máquina
            if ($pl7) {                                     // PL7
                $estado = ST5;
                $p['marcador']['perdidas']++;               // AC11
                fin($p, 'Ha ganado la máquina');
            } elseif ($pl8) {                               // !PL7 && PL8
                $estado = ST6;
                $p['marcador']['empates']++;                // AC8
                fin($p, 'Empate');
            } else {                                        // !PL7 && !PL8
                $estado = ST2;
                es2($p);
            }
            break;
        // ... T5, T6, T7
    }
    // Evento interno: si la máquina tiene el turno, se atiende ya, en esta misma petición
    if ($estado === ST3) {
        $transicion = $automata[ST3][EE_INTERNO];
        $interno = true;
    }
} while ($interno);
```

Debes conocer

Al leer el estado de la página, el programa **rechaza** ST3 (lo trata como ST0): ese estado no puede venir del navegador, porque nunca se le envía. Es la misma idea que las celdas `---` de la tabla, pero para un estado entero.

6.- Pruebas

Como la máquina juega siempre igual, se pueden preparar partidas completas y comprobar cada tablero (31 peticiones):

Partida	Jugadas (jugador → máquina)	Caminos	Resultado
X, empieza el jugador	0→4, 8→2, (4 ocupada), 6→3, 7	T3 → T4 (sigue) ×3, T3 `!PL4`, T3 `PL5`	gana el jugador (la máquina no puede tapar la doble amenaza)
revancha	4→0, 1→7, 2→6, 3→8	T5, T3 → T4 ×3, T4 `PL7`	gana la máquina
revancha	4→0, 2→6, 3→5, 8→1, 7	T3 → T4 ×4, T3 `!PL5 && PL6`	empate (el jugador llena el tablero)
O, empieza la máquina	(máquina 4), 0→2, 6→3, 5→8, 1→7	T2 `PL2` → T4, ..., T4 `!PL7 && PL8`	empate (la máquina llena el tablero)
revancha y Abandonar	(máquina 4)	T5 `PL2` → T4, T6	perdida; marcador 1 / 2 / 2
tablero `XXX------` en el campo oculto	—	T3 `!PL3`	"La partida no era válida"
`estado=3` en la URL	—	ST0 → T1	configuración

7.- Para practicar

- Añade un **nivel fácil** en el que la máquina juegue en una casilla libre al azar. ¿Cambia el autómata o solo la acción AC9? ¿Siguen valiendo las pruebas?
- Haz que la máquina "piense" en **dos pasos internos**: primero decidir y mostrar "La máquina está pensando..." y luego jugar. ¿Cuántos estados transitorios harían falta? ¿Tiene sentido si todo ocurre en la misma petición?
